

Primera Parte del Trabajo final

Teoría de la Computación

Abril 2026

Sobre la entrega:

El uso de Inteligencia Artificial Generativa no está permitida para ningún ejercicio de esta parte de la entrega.

La fecha de entrega límite será el último día de clases (26/6 23:55) por Aulas.

En caso de dudas o preguntas, se deberán preguntar por un foro, que oportunamente será habilitado en Aulas.

Se permitirán entregas de **hasta tres estudiantes del mismo dictado**, siempre que todos suban su entrega a aulas.

Esta entrega tendrá una defensa que se realizará junto al parcial

Objetivo

El objetivo de esta parte es poder desarrollar en Haskell y en λ funciones para resolver y verificar los siguientes problemas.

Problemas a estudiar

A: SAT

Dada una fórmula booleana en forma normal conjuntiva (CNF), decidir si existe una asignación de valores de verdad a sus variables que haga que la fórmula sea verdadera.

B: Planificación cíclica de tareas con restricciones

Una empresa de mantenimiento industrial debe programar una rutina periódica de inspección sobre un conjunto de estaciones distribuidas en una planta. Cada estación requiere ser visitada exactamente una vez por ciclo operativo, y las transiciones entre estaciones tienen costos asociados (por ejemplo, tiempo o consumo energético).

Además, por razones técnicas y de seguridad, existen ciertas dependencias entre tareas y restricciones locales sobre el orden en que pueden realizarse.

Sea una instancia $\mathcal{I} = (V, E, w, \mathcal{P}, \mathcal{X}, k)$, donde:

- V es un conjunto finito de estaciones a inspeccionar,
- $E \subseteq V \times V$ es el conjunto de transiciones permitidas entre estaciones,
- $w : E \rightarrow R_{\geq 0}$ asigna un costo a cada transición,
- $\mathcal{P} \subseteq V \times V$ es un conjunto de restricciones de precedencia,
- $\mathcal{X} \subseteq V \times V$ es un conjunto de exclusiones locales,
- $k \in R_{\geq 0}$ es un umbral máximo de costo.

El problema es: Determinar si existe una secuencia cíclica de estaciones:

$$(v_1, v_2, \dots, v_n, v_1)$$

tal que:

1. **Cobertura total:** cada elemento de V aparece exactamente una vez en la secuencia.
2. **Consistencia de transiciones:** para todo $i = 1, \dots, n$,

$$(v_i, v_{i+1}) \in E,$$

donde $v_{n+1} = v_1$.

3. **Restricciones de precedencia:** para todo $(u, v) \in \mathcal{P}$, si u aparece en la secuencia, entonces v aparece en una posición posterior dentro del recorrido cíclico.
4. **Exclusiones locales:** para todo $(u, v) \in \mathcal{X}$, no ocurre que u sea seguido inmediatamente por v .
5. **Cota de costo:** el costo total del recorrido satisface

$$\sum_{i=1}^n w(v_i, v_{i+1}) \leq k.$$

Este problema modela la planificación de una rutina cíclica eficiente bajo restricciones operativas, donde se busca determinar la existencia de un plan factible que respete tanto dependencias técnicas como limitaciones de costo.

Ejercicio 1: Verificadores y solucionadores en Haskell

1.1 Representación de dominios y soluciones

Definir en Haskell los siguientes tipos:

```
type DomA = undefined
type SolA = undefined
```

```
type DomB = undefined
type SolB = undefined
```

1.2 Verificadores en tiempo polinomial

Implementar las siguientes funciones en Haskell:

```
verifyA :: (DomA, SolA) -> Bool  
verifyB :: (DomB, SolB) -> Bool
```

Justificar formalmente que ambas funciones pueden evaluarse en tiempo polinomial respecto al tamaño de la entrada.

1.3 Resolución en tiempo exponencial

Implementar funciones que devuelvan una solución si existe, utilizando un algoritmo de exploración:

```
solveA :: DomA -> SolA  
solveB :: DomB -> SolB
```

Calcule los órdenes correspondientes.

Estas funciones no deben ser eficientes; su objetivo es ilustrar que la solución puede encontrarse por fuerza bruta y ser verificada en tiempo polinomial.

1.4 Ejemplos

Incluya ejemplos de las funciones realizadas, e indique si los resultados obtenidos son correctos.

Ejercicio 2: Verificadores en χ

Para todo este ejercicio, podrá utilizar macros siempre que las especifique y las programe

2.1 Verificadores en χ

Inspirándose en los verificadores que programó en Haskell en la sección 1.2, prográmelos también en χ puro y embebido en Haskell. Justifique que tienen complejidad polinomial y compare con el orden obtenido del verificador de Haskell

2.2 Ejemplos

Incluya ejemplos de inputs para las funciones realizadas, e indique si los resultados obtenidos son correctos. Para el caso de χ puro usar el intérprete del repo y para el embebido usar el práctico 1 de alguno de los integrantes

Segunda Parte del Trabajo Final

Teoría de la Computación

Mayo 2026

Sobre la entrega

El uso de Inteligencia Artificial Generativa solo está permitido para la investigación de conceptos y para la generación de ejemplos/casos de prueba. Bajo ningún concepto se aceptarán partes completas generadas con IA. Guardar los links de los chats usados (o capturas), que deberán ser incluidos en el informe final

La fecha límite de entrega será informada oportunamente por Aulas.

En caso de dudas o consultas, deberán utilizar el foro correspondiente.

Se permitirán entregas grupales de hasta tres estudiantes del mismo dictado, siempre que todos los integrantes suban su entrega a Aulas.

La entrega podrá incluir una instancia de defensa presencial.

Objetivo

El objetivo de esta tarea es implementar un intérprete del cálculo lambda puro en Haskell utilizando:

- semántica operacional *small-step*,
- estrategia de evaluación eager (*call-by-value*),
- sustitución captura-evitante,
- sharing explícito.

Ejercicio 1: Representación de términos

Se pide definir una representación algebraica de términos del cálculo lambda puro:

$$M ::= x \mid \lambda x.M \mid MN$$

La representación debe permitir modelar:

- variables,
- abstracciones,
- aplicaciones.

En particular, deberá definirse un tipo del estilo:

```
data Expr
```

Ejercicio 2: Variables libres

Se pide implementar una función:

```
fv :: Expr -> [ String ]
```

que calcule el conjunto de variables libres de un término.

Por ejemplo:

```
fv (\x. x y) = [y]
```

```
fv (\x. \y. x) = []
```

Ejercicio 3: Sustitución captura-evitante

Se pide implementar una función:

```
subst :: String -> Expr -> Expr -> Expr
```

donde:

```
subst x n e
```

representa:

$$e[x := n]$$

La implementación deberá:

- evitar captura de variables,
- realizar renombrado α cuando sea necesario.

Por ejemplo, la sustitución:

$$(\lambda y.x)[x := y]$$

NO puede producir:

$$\lambda y.y$$

Ejercicio 4: Semántica operacional

Se pide implementar una función:

`step :: Expr -> Maybe Expr`

que realice exactamente UN paso de evaluación. Formalice también las reglas en formato de DN que utilice.

La estrategia de evaluación deberá ser:

call-by-value

Es decir:

1. evaluar primero la función,
2. luego el argumento,
3. aplicar β -reducción únicamente cuando el argumento sea un valor.

La implementación deberá respetar las siguientes reglas:

β -reducción

$$(\lambda x.M)V \rightarrow M[x := V]$$

solamente cuando V sea un valor.

Aplicación izquierda

$$M_1 \rightarrow M'_1 \implies M_1 M_2 \rightarrow M'_1 M_2$$

Aplicación derecha

$$M_2 \rightarrow M'_2 \implies V M_2 \rightarrow V M'_2$$

Ejercicio 5: Evaluación completa

Se pide implementar una función:

`eval :: Expr -> Expr`

que evalúe repetidamente utilizando `step` hasta alcanzar una forma normal.

Ejercicio 6: Pretty printer

Se pide implementar una función:

`pretty :: Expr -> String`

que imprima expresiones lambda de manera legible. Instanciar el show de Expr para que use esta función

Ejercicio 7: Sharing

Se pide modificar el intérprete para incorporar sharing explícito.

El objetivo es evitar reevaluaciones innecesarias.

Considere el término:

$$(\lambda x.xx)E$$

Sin sharing, el término E puede evaluarse múltiples veces.

Con sharing, el resultado de evaluar E puede almacenarse y reutilizarse.

La implementación deberá:

- identificar expresiones compartidas,
- reutilizar resultados previamente computados,
- evitar trabajo duplicado.

No existe una única implementación correcta. Investigue opciones, justifique la seleccionada

Ejercicio 8: Numerales de Church

Se pide investigar los numerales de Church e implementar tanto en cálculo lambda puro como embebidos en Haskell:

- representación de 0,
- representación de 1,
- representación de 2,
- sucesor,
- suma,
- multiplicación.

Además, deberán incluir ejemplos de evaluación en Λ embebido.

Ejercicio 9: Comparación experimental

Se pide comparar:

- evaluación SIN sharing,
- evaluación CON sharing.

Para ello deberán utilizarse expresiones donde exista trabajo duplicado.

Puede utilizarse:

```
System.CPUTime
```

o bibliotecas de benchmarking como:

```
criterion
```

Debido a la evaluación lazy de Haskell, la medición deberá forzar la evaluación de resultados.

Por ejemplo:

```
result 'seq' return ()
```

Ejercicio 10: Índices de De Bruijn

Se pide investigar:

- qué son los índices de De Bruijn,
- cuál es su importancia,
- qué problemas intentan resolver,
- ventajas y desventajas respecto a representaciones con nombres.

Además, deberán mostrar informalmente cómo se representarían algunos términos lambda utilizando índices de De Bruijn.

Tercera (y última) Parte del Trabajo Final

Teoría de la Computación

Junio 2026

Sobre la entrega

El uso de Inteligencia Artificial Generativa solo está permitido para la investigación de conceptos y para la generación de ejemplos/casos de prueba. Bajo ningún concepto se aceptarán partes completas generadas con IA. Guardar los links de los chats usados (o capturas), que deberán ser incluidos en el informe final

La fecha límite de entrega será informada oportunamente por Aulas.

En caso de dudas o consultas, deberán utilizar el foro correspondiente.

Se permitirán entregas grupales de hasta tres estudiantes del mismo dictado, siempre que todos los integrantes suban su entrega a Aulas.

La fecha de entrega límite será el último día de clases (23:55) por Aulas.

Para la entrega, se deberá citar fuentes externas conforme a lo establecido en los reglamentos de la universidad (Citas IEEE)

Esta entrega tendrá una defensa que se realizará junto al parcial

Modelos de computabilidad

1.1 Verificador de SAT en Imp

Implementar en Imp puro el siguiente programa para la verificación de SAT:

def $VERIFY_{SAT}$ (*clausulas*, *interpretacion*) returns *res*

Que devuelve un booleano correspondiente a la verificación de la instancia *clausulas* sobre la interpretación *interpretacion*. Calcular su complejidad temporal y probar que es polinomial

1.2 Codificación de B en Turing

Definir un alfabeto (Σ) y una codificación de las instancias de DomB como cadenas sobre (Σ). Especificar (no es necesario programar) una Máquina de Turing que resuelva el problema B.

Reducción polinomial

2.1 Codificación de la Reducción en Máquina de Turing

Definir un alfabeto (Σ) y una codificación de las instancias de DomA y DomB como cadenas sobre (Σ). Especificar una Máquina de Turing que, al recibir como entrada una codificación de una instancia de A, produzca como salida la codificación de una instancia de B tal que:

- Si la instancia de A es satisfacible, entonces la instancia de B tiene una solución.
- Si la instancia de A no es satisfacible, entonces la instancia de B no tiene una solución.

2.2 Teorema Cook-Levin

Investigar la prueba del teorema mencionado. Incluir y explicar al menos dos pruebas diferentes para el teorema. (Sugerencia: ver el libro de Sipser)

2.3 Reducción polinomial entre problemas

Demostrar que B es NP-Completo. Definir una estrategia para la reducción de instancias de A a instancias de B:

`reduceAToB :: DomA -> DomB`

No es necesario implementarla, pero sí describirla formalmente.

2.4 Generación de código

Con ayuda de una IA a elección, planteale el esquema de cómo realizó la reducción anterior y pídele que te lo convierta a código del lenguaje de preferencia. Incluir el código. Analizar (sin la IA) ese código (decir qué hubieran hecho similar y qué diferente) y determinar mediante ejemplos y análisis del código si los resultados obtenidos son correctos.

2.5 Ejemplos de instancias de prueba

A continuación se muestran dos instancias simples del problema SAT :

- **Ejemplo 1 (satisfacible):**

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_2 \vee x_3)$$

- **Ejemplo 2 (insatisfacible):**

$$(x_1) \wedge (x_2 \vee x_3) \wedge (\neg x_1)$$

Muestre con qué instancia de B se corresponden y calcule el orden del algoritmo para demostrar que tiene orden polinomial.

Entrega

El contenido de la entrega consistirá en:

- Un archivo Haskell de nombre `solucion.hs` con el código solicitado
- Un documento PDF de nombre `informe.pdf` en el que se desarrollen los ejercicios mencionados. Todas las decisiones tomadas deberán ser debidamente documentados
- Un documento PDF de nombre `articulo.pdf` que cumpla los siguientes requisitos:
 - Documento realizado en L^AT_EX(sugerimos usar Overleaf), siguiendo el template de IEEE
(<https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhnncsfqn>)
 - El informe debe ser autocontenido y autoexplicativo. No es necesario incluir el código completo, pero pseudocódigos pueden ser útiles para guiar la explicación.
 - El artículo abordará únicamente la prueba de que B es NP-completo.